INDIAN INSTITUTE OF TECHNOLOGY, PATNA

MA3206:

AI Assignment

Report – Lab Assignment 5

Submitted To:
Dr. Ranjeet Ranjan Jha

Submitted By:
Aditya Gupta
2301MC51

Aditya Onam
2301MC60

Shakib Akhtar
2301MC49

Mohit Kumar
2301MC47

MDP Policy Evaluation, Value Iteration, and Policy Iteration

1. Problem Formulation

This assignment models a **battery-powered autonomous robot** as a **Markov Decision Process (MDP)** with the objective of maximizing long-term cumulative reward while balancing operational productivity and energy constraints.

The MDP is formally defined as follows:

- **State Space**

  $$S = \{\text{High,Low,Charging}\}$$

  These represent the robot's battery levels.

- **Action Space**

  $$A = \{\text{Search,Wait}\}$$

- **Discount Factor**

  $\gamma = 0.9$, emphasizing long-term reward optimization.

- **Convergence Threshold**

  $\theta = 10^{-6}$, ensuring numerical stability in iterative algorithms.

- **Transition Dynamics & Rewards**

Transition Probabilities P(s' | s, a)

State s	Action a	Next State s'	P(s'|s,a)	Reward R(s,a,s')
High	Search	High	0.7	+4
High	Search	Low	0.3	+4
High	Wait	High	1.0	+1
Low	Search	High	0.4	−3 (recharge penalty)
Low	Search	Low	0.6	+4
Low	Wait	Low	1.0	+1
Charging	Wait	High	1.0	0

Defined explicitly in the table, where:

- Searching in **High** yields high reward but risks battery drop.
- Searching in **Low** carries a recharge penalty risk.
- Charging deterministically restores the battery to **High**.

This structured formulation enables the application of **dynamic programming methods** to compute optimal decision policies.

2. Task 1: Iterative Policy Evaluation

2.1 Objective

The goal is to evaluate a **fixed policy** π by computing the value function $V^{\pi}(s)$, representing the expected cumulative discounted reward from each state.

The Bellman Expectation Equation is:

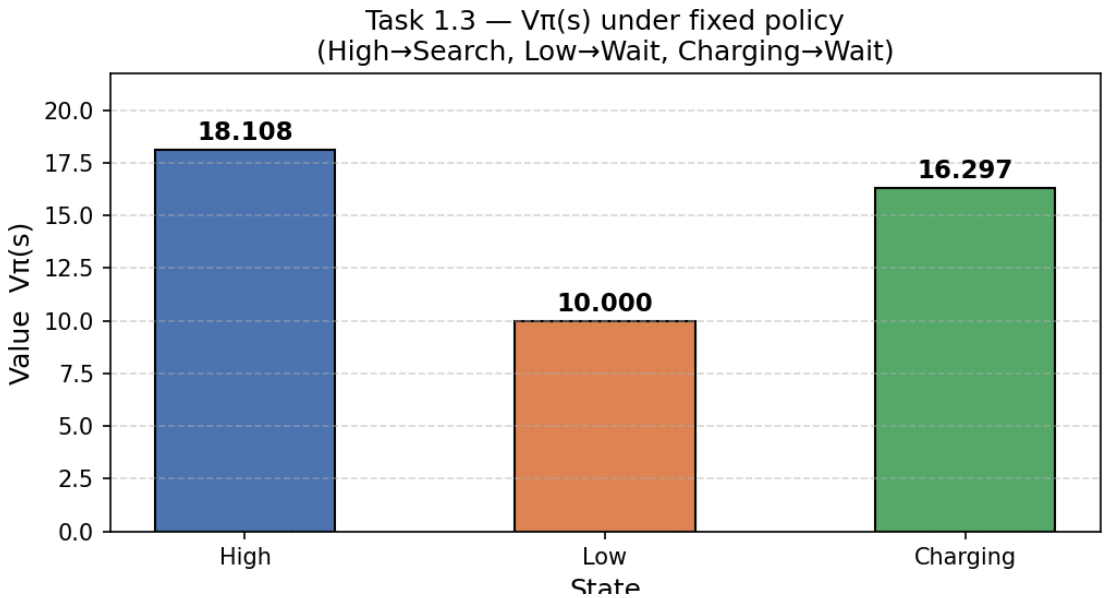
$$V(s) = \sum_{s'} P(s' \mid s, \pi(s))[R(s, \pi(s)) + \gamma V(s')]$$

2.2 Evaluated Policy

$$\pi(\text{High}) = \text{Search}, \pi(\text{Low}) = \text{Wait}, \pi(\text{Charging}) = \text{Wait}$$

2.3 Results

The iterative algorithm converged after **133 iterations**, satisfying the convergence threshold.



The resulting value function is:

- $V^{\pi}(\text{High}) = 18.108$ $\quad V^{\pi}(\text{Low}) = 10.000$ $\quad V^{\pi}(\text{Charging}) = 16.297$

2.4 Interpretation

- The **High state** yields the largest value due to frequent access to high rewards from searching.
- The **Low state** is conservative under the chosen policy (Wait), leading to lower returns.
- The **Charging state** has no immediate reward but transitions to High, hence inherits moderate value.

3. Task 2: Value Iteration

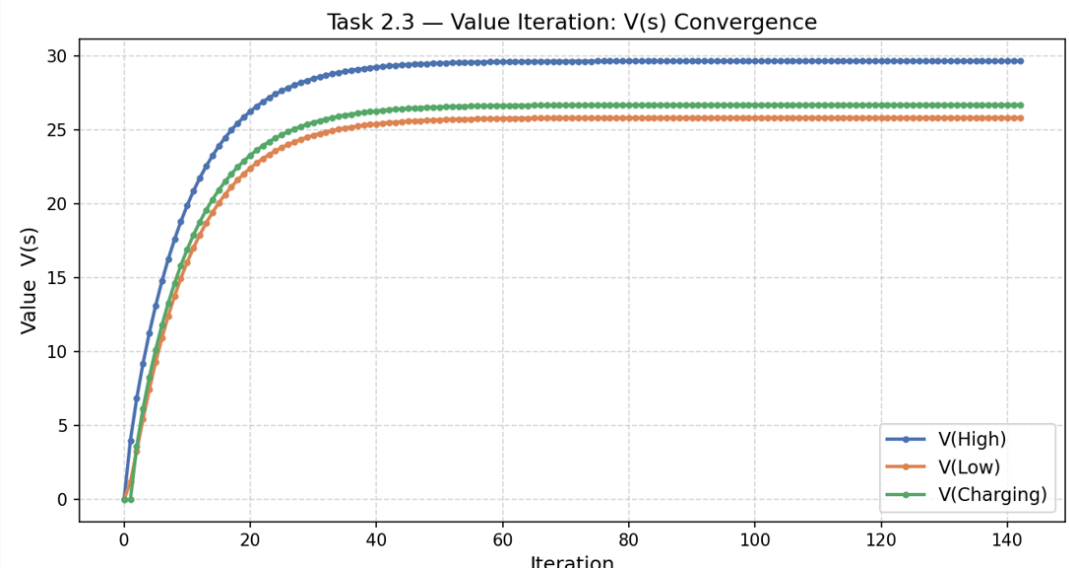
3.1 Objective

Value Iteration directly computes the **optimal value function** $V^*(s)$by repeatedly applying the Bellman Optimality operator.

$$V(s) = \max_a \sum_{s'} P(s' \mid s, a)[R(s, a) + \gamma V(s')]$$

3.2 Results

The algorithm converged after **142 sweeps**.



Optimal Value Function

- $V^*(\text{High}) = 29.644$ $V^*(\text{Low}) = 25.808$ $V^*(\text{Charging}) = 26.679$

Optimal Policy

- High → **Search**
- Low → **Search**
- Charging → **Wait**

3.3 Convergence Behavior

The convergence plot (Task 2.3) shows:

- Smooth, monotonic increase in values
- Gradual stabilization to a fixed point

This reflects the **contraction property** of the Bellman operator under $\gamma < 1$.

4. Task 3: Policy Iteration

4.1 Objective

Policy Iteration alternates between:

1. **Policy Evaluation**
2. **Policy Improvement**

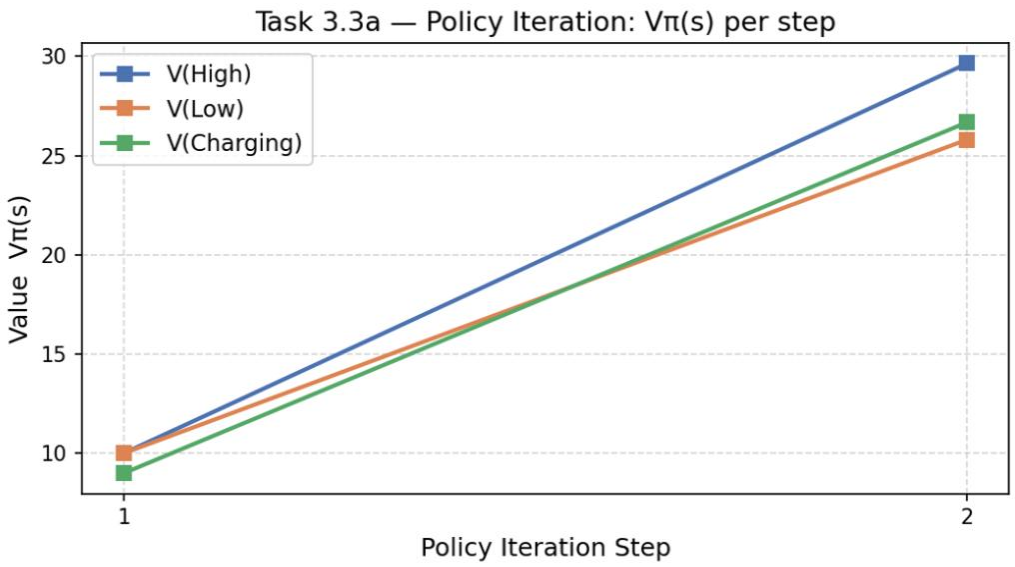
4.2 Execution

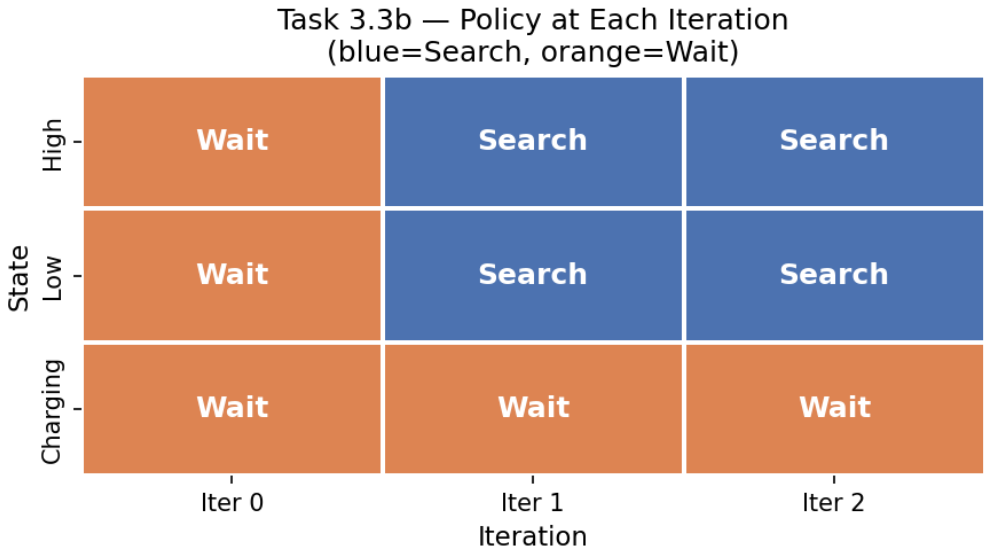
- Initial Policy:

$$\pi_0 = [\text{Wait,Wait,Wait}]$$

4.3 Results

The algorithm converged in **2 outer iterations**, demonstrating high efficiency.





Iteration Summary

- **Iteration 1**
  - Policy: [Wait, Wait, Wait]
  - Value: $[10.0, 10.0, 9.0]$
- **Iteration 2**
  - Policy updated to: [Search, Search, Wait]
  - Value function matched V^*
- Policy stabilized thereafter.

4.4 Visualization Insights

- The **line plot** shows a sharp increase in value after the first improvement step.
- The **policy heatmap** shows immediate transition from all-Wait to optimal configuration.

5. Task 4: Analysis and Interpretation

5.1 Value Iteration vs Policy Iteration

- **Value Iteration:** 142 iterations
- **Policy Iteration:** 2 iterations

Explanation:
Value Iteration performs incremental updates without committing to a policy, requiring many sweeps for convergence.
Policy Iteration, in contrast, performs **full policy evaluation**, leading to highly informed updates and rapid convergence.

5.2 Convergence Behavior

- The Bellman operator is a **contraction mapping**, ensuring convergence to a unique fixed point.
- Values increase monotonically from initialization $V = 0$.
- The **Charging state** stabilizes between Low and High due to deterministic transition to High with zero reward.

5.3 Optimal Policy Interpretation

- **High → Search**
  High immediate reward (+4) outweighs the risk of transitioning to Low due to strong future returns.
- **Low → Search**
  Despite recharge penalty risk, expected reward (~1.2) exceeds waiting (1.0), and access to High state improves long-term gain.
- **Charging → Wait**
  Only feasible action; acts as a deterministic reset to High.

5.4 Practical Insights

This MDP formulation provides a principled alternative to heuristic decision-making in real-world systems such as:

- Autonomous warehouse robots (AGVs)
- Delivery drones
- Battery-constrained IoT agents

Instead of fixed battery thresholds, the MDP framework computes an **optimal trade-off between risk and reward**, ensuring:

- Maximum long-term productivity
- Efficient handling of recharge cycles
- Robust decision-making under uncertainty

Conclusion

This assignment demonstrates how **dynamic programming methods**—Policy Evaluation, Value Iteration, and Policy Iteration—can systematically solve sequential decision-making problems under uncertainty.

Key takeaways:

- All methods converge to the same optimal policy.
- Policy Iteration is significantly faster in practice.
- MDPs provide a mathematically grounded framework for real-world autonomous systems.